



SmartPot

RECORRIDO DEL PROYECTO

De la idea a la **plataforma**

Inicio, análisis, diseño, construcción y pruebas de SmartPot

Cómo nació SmartPot, qué se planeó en la fase de diseño, qué se construyó de verdad, por qué cambió y cómo se comprueba que funciona.



DOCUMENTO

Recorrido del proyecto

VERSIÓN

1.1 · octubre 2026

EQUIPO

SmartPotTech

PROYECTO

smartpot.app



DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Organización	SmartPotTech
Documento	Recorrido del proyecto: inicio, análisis, diseño, construcción y pruebas
Versión	1.1 · octubre 2026
Fuentes	Documentación de diseño del proyecto (requisitos, casos de uso, diagramas UML, acta de constitución, estudio de mercado, matriz de interesados, propuesta de investigación y plan de trabajo), contrastada con el código de los diez repositorios
Documentos hermanos	Documentación técnica : la referencia de la plataforma tal como funciona hoy. Ciclo de vida del software : la lectura crítica de cada etapa, de la formulación a la mejora continua
Cómo leerlo	Cada parte empieza con lo que se planeó y termina con lo que quedó construido. Las tablas de estado dicen, requisito por requisito, qué se cumplió, qué cambió y qué sigue pendiente

CONTENIDO

Contenido

1. Origen del proyecto

- 1.1 La idea
- 1.2 El problema
- 1.3 Visión y justificación
- 1.4 Objetivos
- 1.5 Formulación: 5W + 2H y modelo de negocio

2. Estudio de mercado

3. Interesados

4. Acta de constitución

- 4.1 Alcance y entregables
- 4.2 Hitos
- 4.3 Riesgos y cómo terminaron
- 4.4 Presupuesto

5. Requisitos funcionales

6. Requisitos no funcionales

7. Casos de uso



8. Base de conocimiento

9. Abstracciones del diseño y su implementación

10. Diagramas del diseño actualizados

- 10.1 Clases del dominio
- 10.2 Componentes
- 10.3 Actividad: control automático
- 10.4 Actividad y secuencia: datos históricos
- 10.5 Actividad: enviar un comando

11. Patrones de diseño

12. Interfaz

13. Gestión del proyecto

14. Evolución de la arquitectura

- 14.1 Organización del código
- 14.2 Integración y despliegue continuos
- 14.3 Inteligencia artificial
- 14.4 Telegram, de la maceta a la plataforma
- 14.5 Simulación: Wokwi y cultivos virtuales
- 14.6 El lugar, los switches y los avisos por cultivo

15. Estrategia de pruebas

16. Resultados

- 16.1 Aceptación de los casos de uso
- 16.2 Calidad de los modelos aprendidos

17. Estado final y trabajo futuro

18. Equipo y créditos

PARTE I

Inicio

SECCIÓN 01

Origen del proyecto

En palabras simples

SmartPot empezó como una maceta con un ESP32 que mandaba sus lecturas a un bot de Telegram. En la materia de diseño de software el equipo decidió convertirla en un sistema completo: una aplicación propia, una API, una base de datos y automatizaciones. Con los semestres el proyecto sumó un estudio de mercado, una propuesta de investigación con inteligencia artificial y un plan formal de proyecto, y terminó como la plataforma que hoy corre en smartpot.app.

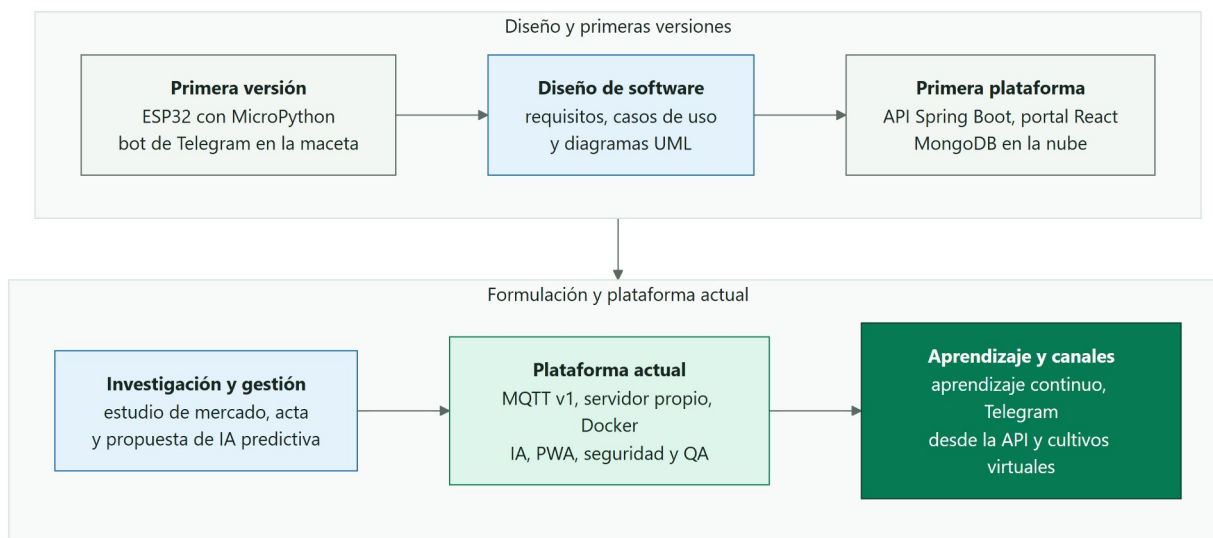


Figura 1 · Etapas del proyecto

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph antes["Diseño y primeras versiones"]
        direction LR
        f1["<b>Primera versión</b><br>ESP32 con MicroPython<br>bot de Telegram en la maceta"]
        f1 --> f2["<b>Diseño de software</b><br>requisitos, casos de uso<br>y diagramas UML"]
        f2 --> f3["<b>Primera plataforma</b><br>API Spring Boot, portal React<br>MongoDB en la nube"]
    end
    subgraph despues["Formulación y plataforma actual"]
        direction LR
        f4["<b>Investigación y gestión</b><br>estudio de mercado, acta<br>y propuesta de IA predictiva"] --> f5["<b>Plataforma actual</b><br>MQTT v1, servidor propio, Docker<br>IA, PWA, seguridad y QA"]
        f5 --> f6["<b>Aprendizaje y canales</b><br>aprendizaje continuo, Telegram<br>desde la API y cultivos virtuales"]
    end
    antes --> despues
    classDef past fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    classDef design fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef now fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef latest fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class f1,f3 past
    class f2,f4 design
    class f5 now
    class f6 latest
```

1.1 La idea

SmartPot fue la idea elegida entre las propuestas de proyecto de un curso de tecnologías de la información: resolvía un problema real, tenía un alcance controlable en un semestre y permitía aplicar a la vez IoT, bases de datos, backend y frontend. Se planteó como un flujo de datos: capturar las lecturas de los sensores, integrarlas y estandarizarlas, automatizar el riego según lo medido y mostrar el estado del cultivo con gráficos y alertas. El [ciclo de vida del software](#) analiza esa formulación en detalle.

1.2 El problema

La hidroponía necesita un control riguroso y constante del pH, los nutrientes, la temperatura, la luz y la humedad. Hecho a mano es ineficiente, depende de que alguien esté presente y detecta tarde los

problemas; las soluciones comerciales son caras y poco flexibles. El diagrama de causa y efecto del proyecto resume el diagnóstico:

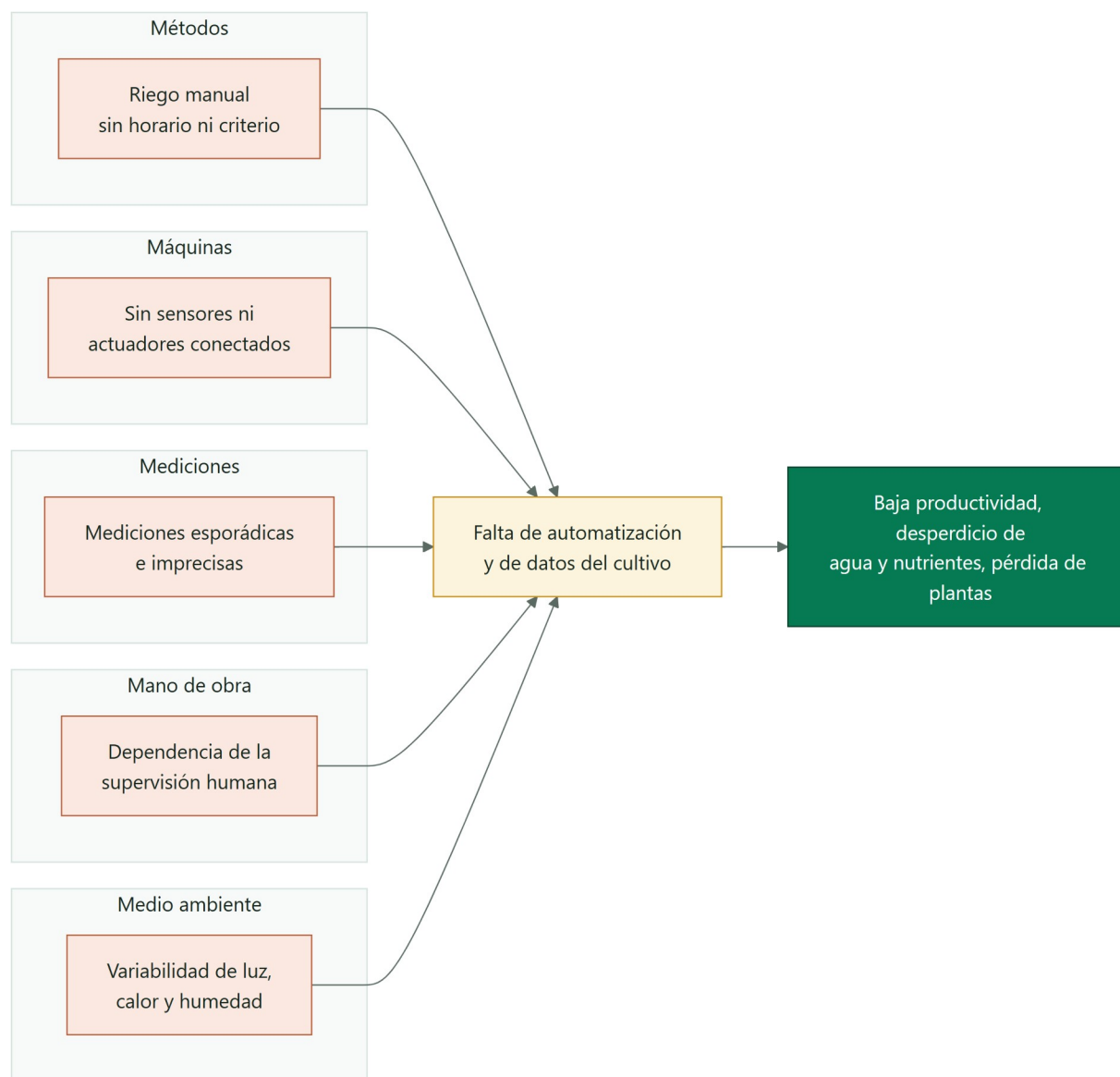


Figura 2 · Causa y efecto del problema



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph metodos["Métodos"]
        m1["Riego manual<br/>sin horario ni criterio"]
    end
    subgraph maquinas["Máquinas"]
        m2["Sin sensores ni<br/>actuadores conectados"]
    end
    subgraph mediciones["Mediciones"]
        m3["Mediciones esporádicas<br/>e imprecisas"]
    end
    subgraph mano["Mano de obra"]
        m4["Dependencia de la<br/>supervisión humana"]
    end
    subgraph ambiente["Medio ambiente"]
        m5["Variabilidad de luz,<br/>calor y humedad"]
    end
    m1 --> raiz["Falta de automatización<br/>y de datos del cultivo"]
    m2 --> raiz
    m3 --> raiz
    m4 --> raiz
    m5 --> raiz
    raiz --> efecto["Baja productividad, desperdicio de<br/>agua y nutrientes, pérdida de plantas"]
    classDef cause fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef root fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef effect fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class m1,m2,m3,m4,m5 cause
    class raiz root
    class efecto effect
```

1.3 Visión y justificación

SmartPot busca ser una solución modular, escalable y de bajo costo para gestionar cultivos hidropónicos, en contextos académicos, domésticos y de pequeña producción. Se alinea con cuatro Objetivos de Desarrollo Sostenible:



ODS	APORTE
2 · Hambre cero	Producir alimentos en poco espacio y con menos recursos, cerca de quien los consume
9 · Industria, innovación e infraestructura	Integrar IoT, datos e inteligencia artificial en la agricultura de precisión
11 · Ciudades y comunidades sostenibles	Agricultura urbana en terrazas, balcones y azoteas
12 · Producción y consumo responsables	Regar y dosificar solo lo necesario

1.4 Objetivos

Objetivo general. Automatizar la gestión de cultivos hidropónicos mediante un sistema inteligente que permita el monitoreo en tiempo real y el análisis de datos, aprovechando los recursos de manera eficiente.

OBJETIVO ESPECÍFICO	CÓMO QUEDÓ
Monitorear de forma remota y continua los parámetros del cultivo	Telemetría MQTT cada pocos segundos y PWA instalable
Optimizar el uso de agua y nutrientes	Riego automático y preventivo; dosificadores como actuadores; regla de bloqueo de nutrientes
Minimizar los errores humanos en la supervisión	Diagnóstico por variable, índice de salud y alertas
Reducir el riesgo de pérdida de plantas	Pronóstico de tendencias y acciones antes de que la variable salga de su rango
Disminuir la dependencia de la supervisión manual	Agente con modo automático y enfriamiento por actuador
Adaptarse a espacios y entornos distintos	Seis especies, cuatro formas (maceta, tubos NFT, torre o balsa), cultivos reales o virtuales y clima real del lugar
Comparar estados históricos y recientes	Historial por cultivo, comparación entre cultivos, estados de operación aprendidos

1.5 Formulación: 5W + 2H y modelo de negocio

PREGUNTA	RESPUESTA
Qué	Un sistema que simula, monitorea y controla las variables críticas de la hidroponía
Para quién	Horticultores urbanos, microproductores, instituciones educativas y la comunidad maker
Por qué	Mantener el pH y los nutrientes en rango es esencial; la automatización reduce desperdicios



PREGUNTA	RESPUESTA
Dónde	Espacios urbanos y periurbanos: terrazas, azoteas, invernaderos pequeños y laboratorios
Cuándo	Por fases: simulación, API, panel y pruebas
Cómo	ESP32 con MicroPython (físico o en Wokwi), backend, aplicación web y base de datos
Cuánto	Casi sin costo en la fase académica; bajo en un prototipo físico

El lienzo del modelo de negocio identificó como propuesta de valor la automatización, el monitoreo en tiempo real, la simulación accesible y la modularidad; como canales, la academia, la web y pilotos educativos.

SECCIÓN 02

Estudio de mercado

SEGMENTO	PERFIL	DEMANDA ESTIMADA INICIAL
Horticultores urbanos y domésticos	Espacios reducidos en áreas metropolitanas	500 a 2000 usuarios
Instituciones educativas	Docentes y estudiantes que necesitan plataformas de bajo costo	50 a 150 instituciones
Makers y comunidad IoT	Desarrolladores que buscan proyectos integrales	200 a 500 usuarios
Microproductores agrotecnológicos	Periferias urbanas y ruralidad cercana	100 a 300 usuarios

OFERTA EXISTENTE	DEBILIDAD	OPORTUNIDAD PARA SMARTPOT
Sistemas comerciales de hidroponía inteligente	Alto costo y poca flexibilidad	Accesibilidad y adaptabilidad
Plataformas IoT genéricas	Requieren conocimiento técnico y no traen el dominio hidropónico	Solución especializada y lista para usar
Desarrollos caseros	Sin soporte ni integración	Interfaz amigable y documentación completa

La estrategia propuesta fue un modelo *freemium* académico en la primera fase y comercial después, con el portal web como canal directo, GitHub como canal de comunidad y las instituciones educativas como multiplicadoras. El estudio concluyó que el mercado existe, que SmartPot ocupa el espacio entre los sistemas caros y los desarrollos caseros complejos, y que los costos de operación del producto mínimo son bajos.

**CÓMO SE REFLEJA HOY**

SmartPot es código abierto con licencia MIT, se publica en GitHub con imágenes en GHCR y Docker Hub, tiene una demo de un solo comando para instituciones y makers, y opera en un servidor propio de bajo costo.

SECCIÓN 03

Interesados

INTERESADO	PODER	INTERÉS	MANEJO	QUÉ NECESITABA
Institución patrocinadora	Bajo	Alto	Mantener informada	Cumplimiento académico, prototipo y documentación
Docente evaluador	Alto	Alto	Administrar de cerca	Entregables del SRS, arquitectura, pruebas y demo
Gerente de proyecto	Alto	Alto	Administrar de cerca	Hitos a tiempo y dentro del presupuesto
Product Owner	Medio	Alto	Administrar de cerca	Backlog priorizado y requisitos claros
Desarrolladores	Bajo	Alto	Mantener informados	Requisitos definidos y un entorno estable
Operadores del jardín	Bajo	Alto	Mantener informados	Monitoreo, alertas y una interfaz intuitiva
Servicios en la nube	Bajo	Bajo	Monitorear	Consumo eficiente, sin costos extra
Comunidad de código abierto	Bajo	Bajo	Monitorear	Repositorio documentado y reutilizable
Feria de emprendimiento	Bajo	Medio	Monitorear	Una demo que muestre viabilidad y potencial

SECCIÓN 04

Acta de constitución

4.1 Alcance y entregables

El acta definió un sistema de tres capas (IoT, backend y frontend) desarrollado en dos fases: análisis y diseño, y desarrollo, pruebas e integración.



ENTREGABLE PLANEADO	ESTADO
Documento de requisitos (SRS)	Requisitos y casos de uso consolidados en este documento, con su estado actual
Diseño del sistema	Diagramas actualizados a la plataforma construida (parte III)
Simulación IoT	Firmware MicroPython en Wokwi (un cultivo real) y cultivos virtuales siempre encendidos
Backend	API REST con JWT, MQTT, automatización y control de dueño
Frontend	PWA con monitoreo, control, panel general, asistente y aprendizaje
Informe de pruebas	Parte V y el workflow de QA de la organización
Manual de usuario	README de cada repositorio, guía de la demo y ayuda dentro de la PWA
Código fuente documentado	Diez repositorios en GitHub con CI, documentación y diagramas

4.2 Hitos

HITO PLANEADO	RESULTADO
Inicio: equipo, repositorios y herramientas	Organización SmartPotTech con un repositorio por componente
Definición de requisitos	RF-001 a RF-017 y RNF-001 a RNF-006
Diseño de arquitectura	Arquitectura por componentes, contrato MQTT v1 y API REST
Simulación IoT funcional	Firmware en Wokwi publicando por MQTT; simulador para demo y QA
Backend core	Lecturas, autenticación JWT y automatización
Frontend web	PWA con monitoreo, control y panel
Pruebas y QA	Pruebas en cada repositorio y QA de extremo a extremo
Entrega final	Plataforma en producción en smartpot.app y documentación

4.3 Riesgos y cómo terminaron

RIESGO	VALORACIÓN	QUÉ PASÓ
Integración entre la simulación y el backend	Alta	Se resolvió con un contrato MQTT v1 explícito, un simulador que lo cumple y una prueba de extremo a extremo que lo recorre en cada cambio
Cambios de alcance	Alta	El alcance creció (IA, panel general, cultivos virtuales, Telegram, ilustración de cada cultivo) sin romper lo existente gracias a la separación por componentes
Curva de aprendizaje	Moderada	Se documentó cada repositorio y se estandarizaron herramientas (uv, pnpm, Maven Wrapper)



RIESGO	VALORACIÓN	QUÉ PASÓ
Deuda técnica	Moderada	Se hizo una reestructuración completa con pruebas, Dependabot y CI en todos los repositorios
Cuotas de servicios gratuitos	Moderada	Se reemplazaron por un servidor propio con Docker, sin depender de capas gratuitas
Seguridad y acceso no autorizado a actuadores	Moderada	Se encontraron y corrigieron rutas sin control de dueño; cuenta MQTT por cultivo, cifrado AES-GCM de claves, límite de peticiones y CodeQL
Saturación por datos IoT	Moderada	MQTT con QoS, una lectura cada 5 s como máximo por cultivo y Redis para límites y caché
Retrasos en la ruta crítica	Alta	El plan se reprogramó con nivelación de recursos (sección 13)

4.4 Presupuesto

El acta asignó 1 000 000 COP: 40 % a infraestructura en la nube, 50 % al tiempo del equipo, 10 % a documentación y presentación, y 0 % a licencias gracias a herramientas abiertas. La plataforma actual mantiene ese espíritu: todo el software es abierto y la operación cabe en un servidor virtual pequeño.



PARTE II

Análisis

SECCIÓN 05

Requisitos funcionales

En palabras simples

Los requisitos se escribieron antes de programar, cuando SmartPot todavía era una idea. La mayoría se cumplió tal cual; algunos cambiaron porque la construcción encontró una forma mejor, y unos pocos quedan como trabajo futuro. Esta tabla es la matriz de trazabilidad: requisito, estado y dónde vive.

REQUISITO	ESTADO	CÓMO QUEDÓ CONSTRUIDO
RF-001 Acceder a la plataforma	Cumplido	Registro, ingreso con JWT y recuperación con enlace de 30 minutos (en lugar de enviar credenciales por correo)
RF-002 Calibrar sensores	Parcial	La escala de cada sensor se ajusta en el firmware; la PWA administra los actuadores y la clave del dispositivo. La calibración remota queda pendiente
RF-003 Visualizar el estado de las plantas	Cumplido	Lecturas frente al rango ideal, índice de salud y panel general de todos los cultivos
RF-004 Regar automáticamente	Cumplido	El agente riega con el modo automático; también de forma preventiva por pronóstico y por lo aprendido
RF-005 Notificar alertas	Cumplido	Alertas en la PWA y por Telegram: cada cultivo elige qué avisa, al instante o en resúmenes, con un resumen diario y chats con los que se comparte
RF-006 Visualizar datos históricos	Cumplido	Historial de 6 h, 24 h o 7 días por variable y exportación CSV
RF-007 Registrar datos de sensores	Cumplido	Cada lectura MQTT se guarda en MongoDB durante un año
RF-008 Configurar parámetros	Replanteado	Los umbrales vienen de la base de conocimiento de seis especies; umbrales propios por cultivo quedan pendientes
RF-009 Automatizar el control de luz	Cumplido	Luz ultravioleta cuando falta de día y apagado en el descanso nocturno; el asistente recomienda sol directo a las especies que lo piden
RF-010 Controlar manualmente	Cumplido	Un switch por actuador que solo acepta órdenes que cambian algo, y órdenes en bloque a varios cultivos



REQUISITO	ESTADO	CÓMO QUEDÓ CONSTRUIDO
RF-011 Visualizar y editar el perfil	Cumplido	Perfil, contraseña, canales de notificación y borrado de la cuenta
RF-012 Generar reportes	Parcial	Resumen estadístico, exportación CSV, análisis de todos los cultivos, aprendizaje por especie y resúmenes programados por Telegram (cada 1 a 24 h y uno diario); los reportes en documento quedan pendientes
RF-013 Gestión integral de plantas	Cumplido	Crear, editar y eliminar cultivos de seis especies, reales o virtuales, con su forma, su lugar y sus actuadores
RF-014 Gestión de nutrientes	Parcial	Se mide el TDS, existe el dosificador como actuador y la regla de bloqueo de nutrientes; la dosificación física espera hardware
RF-015 Conectar con la API	Cumplido	Contrato MQTT v1 para los dispositivos y REST documentado en /docs
RF-016 Visualizar gráficos históricos	Cumplido	Gráficos con la banda ideal y comparación de una variable entre cultivos
RF-017 Determinar el estado general	Cumplido	Índice difuso de 0 a 100 en cinco niveles, más fino que los tres planteados

SECCIÓN 06

Requisitos no funcionales

REQUISITO	ESTADO	CÓMO QUEDÓ
RNF-001 Autenticación en dos pasos	Pendiente	Mitigado con BCrypt de costo 12, límite de peticiones y tokens de recuperación de un solo uso
RNF-002 Paleta orientada a la naturaleza	Cumplido	Paleta SmartPot: verdes hoja, azul agua, sol y arcilla, validada para daltonismo en las gráficas
RNF-003 Sincronización en tiempo real	Cumplido	El dispositivo publica por MQTT y la API guarda en segundos; la PWA se actualiza cada 10 a 30 s. Un canal en vivo hacia la PWA queda como mejora
RNF-004 Soporte para distintos dispositivos	Cumplido	PWA adaptable e instalable en Android, iOS y escritorio
RNF-005 Navegación clara	Cumplido	Navegación lateral en escritorio e inferior en el teléfono, textos simples en español
RNF-006 Personalización	Pendiente	El tema claro con la paleta de marca es fijo; temas y ajustes de visualización quedan pendientes



SECCIÓN 07

Casos de uso

Los ocho casos de uso del diseño se mantienen; la construcción sumó cuatro.

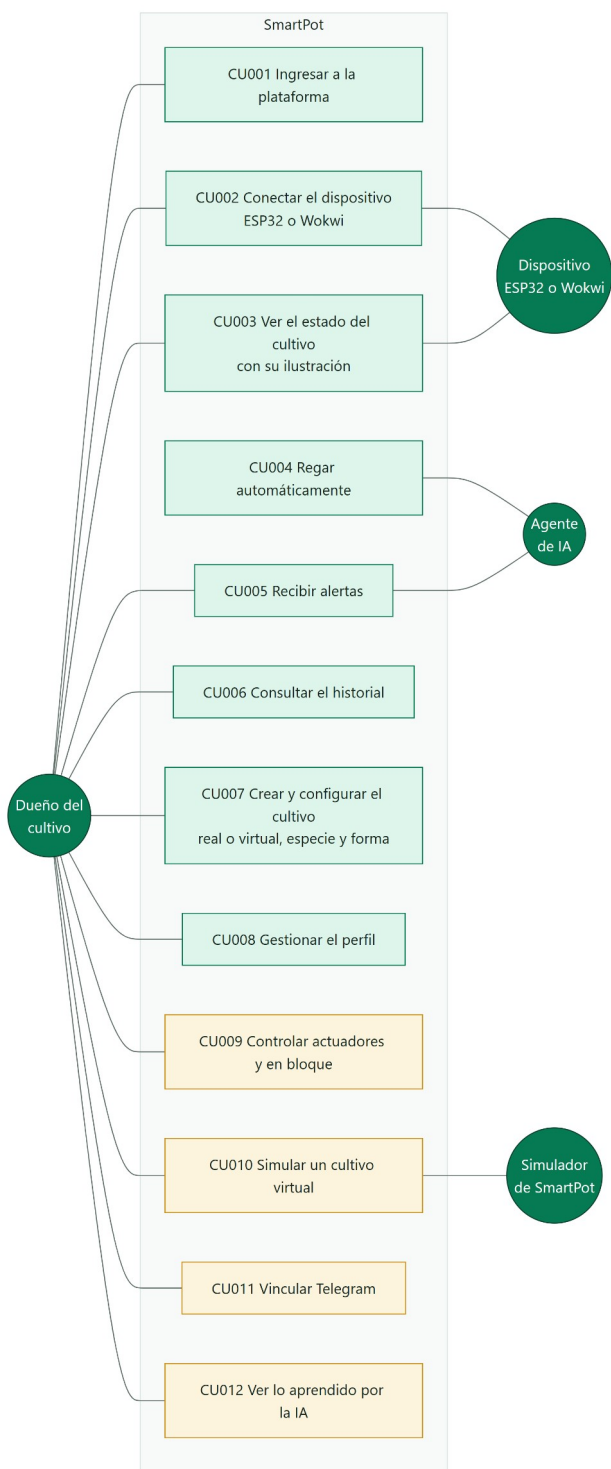


Figura 3 · Casos de uso de SmartPot



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    persona(("Dueño del<br/>cultivo"))
    subgraph sistema["SmartPot"]
        direction TB
        cu1["CU001 Ingresar a la plataforma"]
        cu2["CU002 Conectar el dispositivo<br/>ESP32 o Wokwi"]
        cu3["CU003 Ver el estado del cultivo<br/>con su ilustración"]
        cu4["CU004 Regar automáticamente"]
        cu5["CU005 Recibir alertas"]
        cu6["CU006 Consultar el historial"]
        cu7["CU007 Crear y configurar el cultivo<br/>real o virtual, especie y forma"]
        cu8["CU008 Gestionar el perfil"]
        cu9["CU009 Controlar actuadores y en bloque"]
        cu10["CU010 Simular un cultivo virtual"]
        cu11["CU011 Vincular Telegram"]
        cu12["CU012 Ver lo aprendido por la IA"]
    end
    maceta(("Dispositivo<br/>ESP32 o Wokwi"))
    simulador(("Simulador<br/>de SmartPot"))
    agente(("Agente<br/>de IA"))
    persona --- cu1
    persona --- cu2
    persona --- cu3
    persona --- cu5
    persona --- cu6
    persona --- cu7
    persona --- cu8
    persona --- cu9
    persona --- cu10
    persona --- cu11
    persona --- cu12
    cu2 --- maceta
    cu3 --- maceta
    cu10 --- simulador
    cu4 --- agente
    cu5 --- agente
    classDef actor fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef base fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef nuevo fill:#FDF4DD,stroke:#C98D12,color:#17261F
    class persona,maceta,simulador,agente actor
    class cu1,cu2,cu3,cu4,cu5,cu6,cu7,cu8 base
    class cu9,cu10,cu11,cu12 nuevo
```



CASO DE USO	FLUJO PRINCIPAL HOY	EXCEPCIONES CUBIERTAS
CU001 Ingresar a la plataforma	Correo y contraseña; «Mantener sesión iniciada»	Credenciales inválidas; recuperación por enlace
CU002 Conectar el dispositivo (ESP32 o Wokwi)	Crear un cultivo real y seguir la guía: circuito, firmware y <code>config.py</code> , o el proyecto de Wokwi	Clave mostrada una sola vez; rotación si se pierde
CU003 Ver el estado del cultivo	Panel general, resumen con el rango ideal e ilustración del cultivo sobre todas sus secciones, con su forma, su especie y cada actuador	Sin lecturas: aviso para conectarlo; desconectado: no se ilustra y se explica cómo conectarlo
CU004 Regar automáticamente	El agente riega con el modo automático y avisa	Posible falla de sensor: no actúa; enfriamiento por actuador
CU005 Recibir alertas	Notificaciones en la PWA y en Telegram	Chat bloqueado: el canal se pausa
CU006 Consultar el historial	Historial por variable y exportación CSV	Sin datos en el periodo
CU007 Crear y configurar el cultivo	Real o virtual (no cambia después), nombre, especie, forma y modo automático	Validación de campos en español; cambiar el tipo se rechaza
CU008 Gestionar el perfil	Datos, contraseña y borrado de la cuenta	Contraseña actual incorrecta
CU009 Controlar actuadores y en bloque	Encender o apagar un actuador o la misma orden en varios cultivos	Cultivo sin ese actuador: se informa por cultivo
CU010 Simular un cultivo virtual	Crearlo virtual con clima real de un lugar, medidores manuales o día y noche; pausarlo y reanudarlo	Sin ubicación en modo clima; límite de 5 por cuenta; no entrega credenciales
CU011 Vincular Telegram	Código de un solo uso desde el perfil y <code>/start</code> en el bot	Código vencido o usado
CU012 Ver lo aprendido por la IA	Página Aprendizaje y sección del asistente	Especie sin datos suficientes: se explica qué falta

SECCIÓN 08

Base de conocimiento

La investigación temática de la fase de diseño (qué es un jardín hidropónico, sus partes, el ESP32, Wokwi y las métricas de lechuga y tomate) se convirtió en la base de conocimiento del asistente, ampliada a seis especies y llevada a la escala de los sensores de la maceta.

VARIABLE	MÉTRICA INVESTIGADA	CÓMO QUEDÓ
pH	5.5 a 6.5 para lechuga y tomate	Se conserva; fresa, espinaca y pimentón con sus propios rangos



VARIABLE	MÉTRICA INVESTIGADA	CÓMO QUEDÓ
Nutrientes	Lechuga 800 a 1700 ppm; tomate con CE de 2.0 a 3.5 mS/cm	Todo en ppm con factor 700 (lechuga 560 a 840, tomate 1400 a 2800)
Temperatura del aire	Lechuga 18 a 24 °C; tomate 22 a 28 °C de día	Lechuga 15 a 22 °C y tomate 20 a 28 °C, con tolerancia de 5 °C
Humedad relativa	50 a 70 % y 60 a 70 %	Lechuga 50 a 70 %, tomate 60 a 80 %
Luz	Lechuga de 20 000 a 40 000 lux	En la escala relativa del sensor (0 a 2000) y con descanso nocturno de 22:00 a 6:00
Humedad del sustrato	No estaba en la investigación inicial	Se agregó como variable propia, clave para el riego

PARTE III

Diseño

SECCIÓN 09

Abstracciones del diseño y su implementación

En palabras simples

El diseño imaginó una maceta que decidía por sí misma: leía sus sensores, evaluaba la salud del cultivo y encendía los actuadores. La construcción movió esa inteligencia a la plataforma: el dispositivo mide, publica y obedece; la API y la IA deciden. Así un dispositivo barato se beneficia de todo lo que la plataforma aprende.

ABSTRACCIÓN DEL DISEÑO	IMPLEMENTACIÓN
Sensor con Brightness, Atmosphere, PH y TDS	Firmware: ADCSensor con LightSensor, PHSensor, TDSSensor y SoilMoistureSensor, y AtmosphereSensor (DHT22) que entrega temperatura y humedad por separado en lugar de un valor compuesto
Actuador con WaterPump y UVLight	Firmware: Actuator y ActuatorBank con apagado al cumplir la duración. API: Actuator con seis tipos (bomba, luz ultravioleta, ventilador, humidificador y dosificadores de pH y nutrientes)
LCD Display	Se conserva en el firmware (LCDDisplay 20×4 por I2C) como respaldo local cuando no hay conexión
DeviceController	Firmware: ciclo principal y SmartPotClient (MQTT con TLS). La toma de decisiones pasó al agente de la plataforma
API con consulta periódica de solicitudes	Tópico <code>commands</code> con QoS 1 y confirmación en <code>commands/ack</code> ; el «identificador de última solicitud» se volvió el id del comando y su máquina de estados
Cultivo.evaluarSalud()	Crop en la API y el asistente de IA: diagnóstico por variable, sistema experto, índice difuso y pronóstico
Usuario	User con rol y datos básicos
Sesión	JWT sin estado en el servidor; recuperación con PasswordResetToken de un solo uso
Notificación	Notification en la PWA y ChannelLink para reenviarla a Telegram
Registro Cultivo e Historial Cultivo	Reading con Measures; el historial son consultas por rango, resúmenes y series agregadas en MongoDB



ABSTRACCIÓN DEL DISEÑO	IMPLEMENTACIÓN
FabricaCultivos	Perfiles por especie de la base de conocimiento, compartidos por todos los cultivos de esa especie

SECCIÓN 10

Diagramas del diseño actualizados

Los diagramas de la fase de diseño describían una aplicación distinta de la construida. Estos son sus equivalentes actuales; la [documentación técnica](#) trae además la arquitectura general, el flujo de una lectura, los estados de un comando, el modelo de datos, el despliegue y las redes.

10.1 Clases del dominio

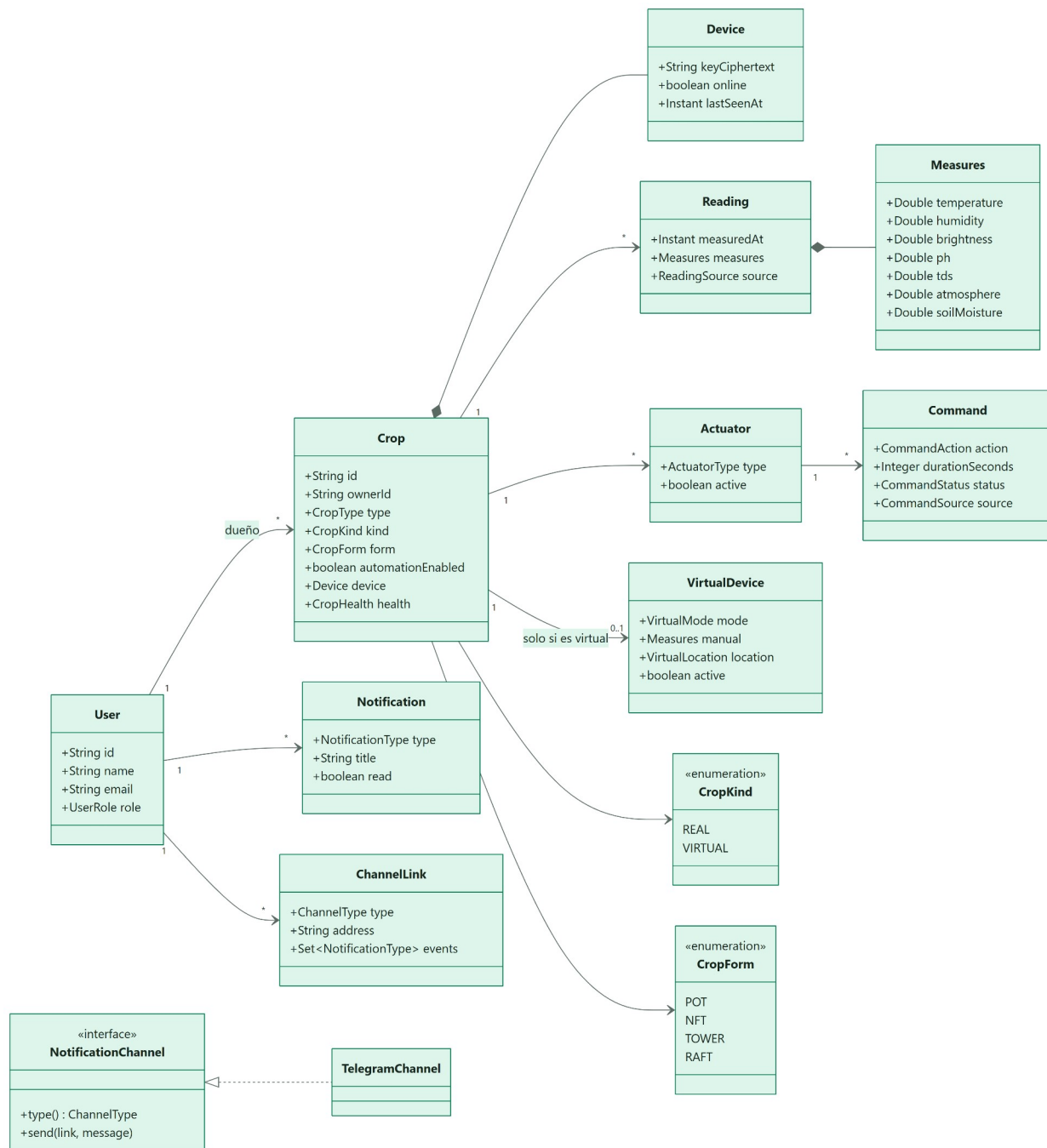


Figura 4 · Clases del dominio



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
classDiagram
    direction LR
    class User {
        +String id
        +String name
        +String email
        +UserRole role
    }
    class Crop {
        +String id
        +String ownerId
        +CropType type
        +CropKind kind
        +CropForm form
        +boolean automationEnabled
        +Device device
        +CropHealth health
    }
    class Device {
        +String keyCiphertext
        +boolean online
        +Instant lastSeenAt
    }
    class Reading {
        +Instant measuredAt
        +Measures measures
        +ReadingSource source
    }
    class Measures {
        +Double temperature
        +Double humidity
        +Double brightness
        +Double ph
        +Double tds
        +Double atmosphere
        +Double soilMoisture
    }
    class Actuator {
        +ActuatorType type
        +boolean active
    }
    class Command {
```



```
+CommandAction action
+Integer durationSeconds
+CommandStatus status
+CommandSource source
}
class Notification {
  +NotificationType type
  +String title
  +boolean read
}
class ChannelLink {
  +ChannelType type
  +String address
  +Set~NotificationType~ events
}
class VirtualDevice {
  +VirtualMode mode
  +Measures manual
  +VirtualLocation location
  +boolean active
}
class CropKind {
  <<enumeration>>
  REAL
  VIRTUAL
}
class CropForm {
  <<enumeration>>
  POT
  NFT
  TOWER
  RAFT
}
class NotificationChannel {
  <<interface>>
  +type() ChannelType
  +send(link, message)
}
class TelegramChannel
User "1" → "*" Crop : dueño
Crop *-- Device
Crop "1" → "*" Reading
Reading *-- Measures
Crop "1" → "*" Actuator
Actuator "1" → "*" Command
User "1" → "*" Notification
User "1" → "*" ChannelLink
Crop "1" → "0..1" VirtualDevice : solo si es virtual
Crop → CropKind
Crop → CropForm
NotificationChannel <|.. TelegramChannel
```




10.2 Componentes

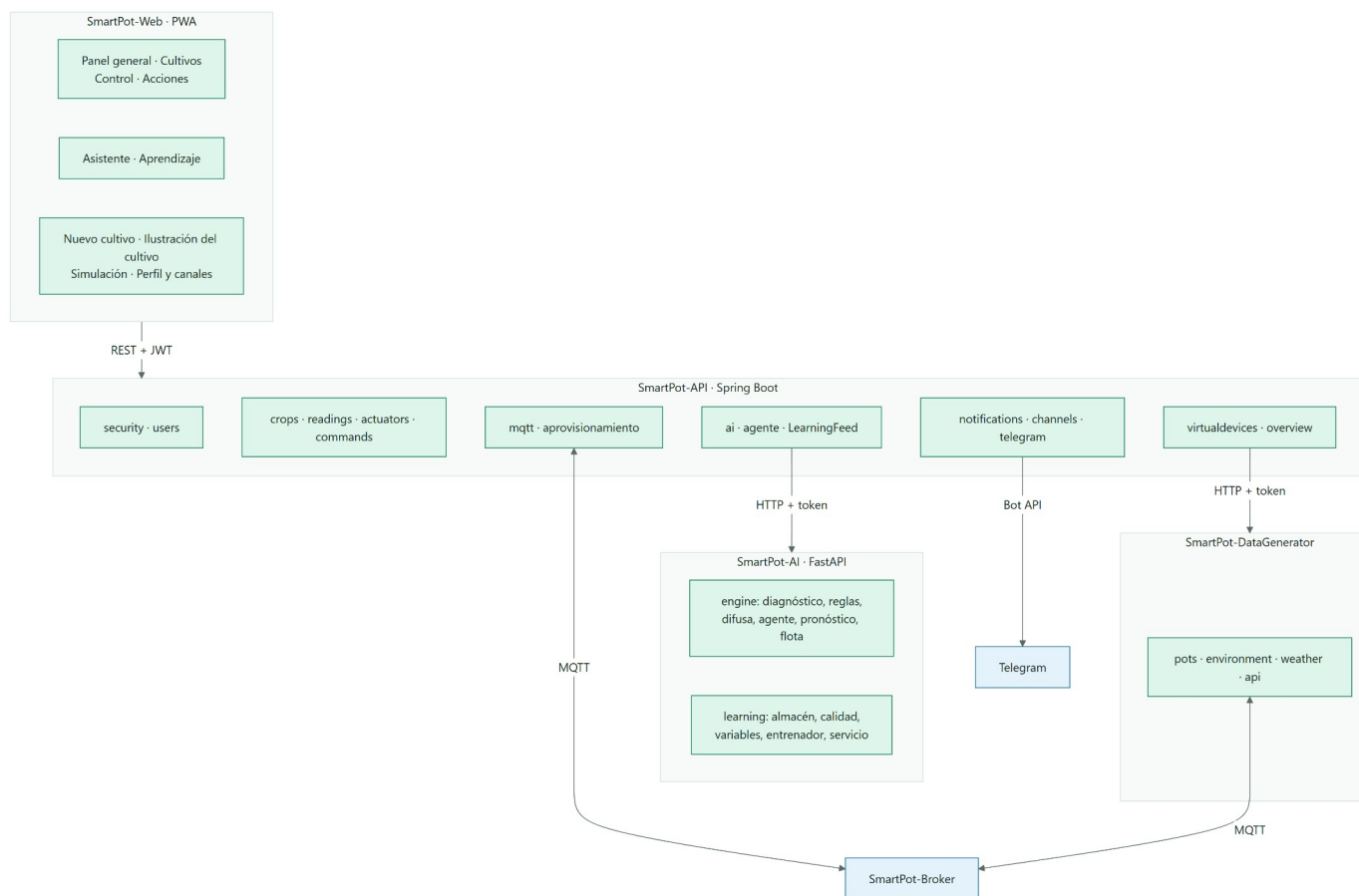


Figura 5 · Componentes y paquetes



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}%%
flowchart TB
    subgraph web["SmartPot-Web · PWA"]
        w1["Panel general · Cultivos<br/>Control · Acciones"]
        w2["Asistente · Aprendizaje"]
        w3["Nuevo cultivo · Ilustración del cultivo<br/>Simulación · Perfil y canales"]
    end
    subgraph api["SmartPot-API · Spring Boot"]
        a1["security · users"]
        a2["crops · readings · actuators · commands"]
        a3["mqtt · aprovisionamiento"]
        a4["ai · agente · LearningFeed"]
        a5["notifications · channels · telegram"]
        a6["virtualdevices · overview"]
    end
    subgraph ai["SmartPot-AI · FastAPI"]
        i1["engine: diagnóstico, reglas,<br/>difusa, agente, pronóstico, flota"]
        i2["learning: almacén, calidad,<br/>variables, entrenador, servicio"]
    end
    subgraph sim["SmartPot-DataGenerator"]
        s1["pots · environment · weather · api"]
    end
    web -->|"REST + JWT"| api
    a3 <-->|"MQTT"| broker["SmartPot-Broker"]
    s1 <-->|"MQTT"| broker
    a4 -->|"HTTP + token"| ai
    a6 -->|"HTTP + token"| sim
    a5 -->|"Bot API"| tg["Telegram"]
    classDef box fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef ext fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    class w1,w2,w3,a1,a2,a3,a4,a5,a6,i1,i2,s1 box
    class broker,tg ext
```

El diagrama de paquetes del diseño agrupaba **GestorUsuario**, **GestorCultivo**, **API** y **Jardin**. Hoy cada paquete de la API es un dominio con la misma organización por capas (**controller**, **service**, **repository**, **mapper** y **model**), y el jardín vive en dos repositorios: el firmware y el simulador.



10.3 Actividad: control automático

El diagrama de actividades del diseño («Control automático») planteaba leer los sensores, comparar con los límites, generar la alerta y el comando al actuador y guardar su estado. La construcción conserva ese flujo y le agrega la evaluación del asistente, el enfriamiento, la confirmación de la maceta y el aprendizaje.

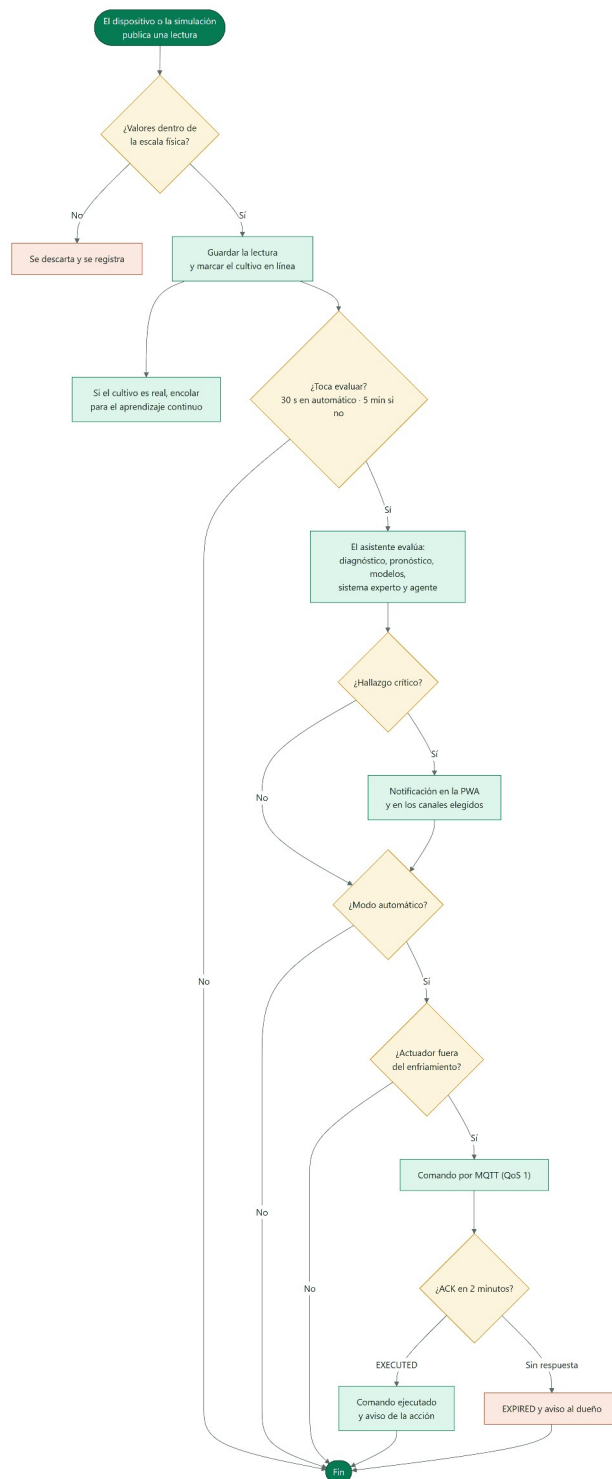


Figura 6 · Actividad del control automático



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    inicio(["El dispositivo o la simulación publica una lectura"]) --> valida{"¿Valores dentro de la escala física?"}
    valida -->|"No"| descarta["Se descarta y se registra"]
    valida -->|"Sí"| guarda["Guardar la lectura y marcar el cultivo en línea"]
    guarda --> aprende["Si el cultivo es real, encolar para el aprendizaje continuo"]
    guarda --> evalua{"¿Toca evaluar? 30 s en automático · 5 min si no"}
    evalua -->|"No"| fin(["Fin"])
    evalua -->|"Sí"| ia["El asistente evalúa diagnóstico, pronóstico, modelos, sistema experto y agente"]
    ia --> critico{"¿Hallazgo crítico?"}
    critico -->|"Sí"| alerta["Notificación en la PWA y en los canales elegidos"]
    critico -->|"No"| auto
    alerta --> auto{"¿Modo automático?"}
    auto -->|"No"| fin
    auto -->|"Sí"| enfria{"¿Actuador fuera del enfriamiento?"}
    enfria -->|"No"| fin
    enfria -->|"Sí"| comando["Comando por MQTT (QoS 1)"]
    comando --> ack{"¿ACK en 2 minutos?"}
    ack -->|"EXECUTED"| ok["Comando ejecutado y aviso de la acción"]
    ack -->|"Sin respuesta"| vence["EXPIRED y aviso al dueño"]
    ok --> fin
    vence --> fin
    classDef start fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef step fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef ask fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef bad fill:#FBE9E1,stroke:#B85A38,color:#17261F
    class inicio,fin start
    class guarda,aprende,ia,alerta,comando,ok step
    class valida,evalua,critico,auto,enfria,ack ask
    class descarta,vence bad

```

10.4 Actividad y secuencia: datos históricos

El módulo elegido para la primera implementación fue **Datos históricos**. Su diagrama de actividades y su secuencia pedían verificar que la persona tuviera cultivo, que existieran registros, mostrar una tabla y permitir cambiar a gráfica. Hoy: la PWA avisa si no hay cultivos o si aún no hay lecturas, el historial muestra un gráfico por variable con la banda ideal, se elige el periodo (6 h, 24 h o 7 días) y se exporta a CSV. La librería de gráficos pasó de D3.js a Recharts.



10.5 Actividad: enviar un comando

El diseño validaba la sesión, verificaba que hubiera cultivo asociado, dejaba el comando en una cola, lo enviaba al broker y esperaba el estado. Se construyó igual: control de dueño, comando **PENDING**, publicación con QoS 1, **SENT**, confirmación **EXECUTED** o **FAILED**, y **EXPIRED** con aviso si la maceta no responde en 2 minutos.

SECCIÓN 11

Patrones de diseño

PATRÓN PLANEADO	DÓNDE QUEDÓ
Abstract Factory para crear sensores	Se simplificó a una jerarquía: <code>ADCSensor</code> concentra la lectura y el mapeo de escala; cada sensor solo define su rango. En MicroPython una fábrica añadía memoria sin beneficio
Facade (<code>print_table</code> y <code>send_msg</code>)	<code>utils.py</code> sigue ocultando el formato de la consola y <code>smartpot_client.py</code> oculta MQTT, los tópicos y las cargas. El envío a Telegram salió de la maceta y ahora vive en la API
Flyweight (cultivo compartido y registros)	Los perfiles por especie son el estado compartido; las lecturas son el estado propio de cada cultivo

La construcción sumó otros patrones:

PATRÓN	USO
Strategy	<code>NotificationChannel</code> : cada canal (hoy Telegram) es una implementación intercambiable
Observer	Eventos de Spring: lectura registrada, notificación creada, cultivo eliminado y clave rotada; cada módulo reacciona sin acoplarse
Gateway y Null Object	<code>MqttGateway</code> con la implementación de Paho y una deshabilitada para pruebas
Repository, DTO y Mapper	Persistencia y contratos separados de las entidades en cada dominio
Sistema de producción	Reglas del sistema experto con prioridad y encadenamiento hacia adelante
Pipeline y campeón y retador	Escalado y modelo en un solo <code>Pipeline</code> ; un modelo nuevo solo reemplaza al vigente si lo mejora

SECCIÓN 12

Interfaz

Los prototipos de Figma definieron las pantallas principales y la navegación. La PWA las conserva (ingreso, panel, detalle del cultivo, historial, perfil y alertas) y suma el panel general, el control general, el



centro de acciones, la página de aprendizaje, la ilustración de cada cultivo en su lugar, la pestaña de simulación de los cultivos virtuales y las aplicaciones de notificación del perfil. El requisito de una paleta orientada a la naturaleza se cumplió con la paleta SmartPot, que también usan estos documentos.

PARTE IV

Construcción

SECCIÓN 13

Gestión del proyecto

El plan se armó con Scrum en sprints de dos semanas, una estructura de desglose del trabajo de cuatro niveles, estimaciones PERT y un cronograma con ruta crítica.

ANÁLISIS	RESULTADO
Ruta crítica	Continúa desde el acta de constitución hasta el cierre; los puntos más sensibles fueron la especificación de contratos de la API, el SRS y las pruebas de integración
Recursos	Hoja con personas, equipos, plataformas en la nube y herramientas, con códigos para la trazabilidad
Nivelación	Se eliminaron las sobreasignaciones del equipo: el cierre pasó al 10 de junio de 2026, con 56 días de duración
Lección	Con cuatro personas efectivas, mantener la fecha original exigía cargas diarias imposibles; la reprogramación fue una corrección del plan, no un error

SECCIÓN 14

Evolución de la arquitectura

ASPECTO	DISEÑO INICIAL	CONSTRUIDO	POR QUÉ
Comunicación del dispositivo	Bot de Telegram en el ESP32; luego consulta periódica de solicitudes por HTTP	MQTT v1 con TLS, QoS 1, ACK, estado retenido y última voluntad	Conexión liviana y bidireccional, y saber si el cultivo está en línea
Base de datos	H2 o MySQL con JPA en el acta; esquemas de Mongoose en el modelo	MongoDB 8 con validadores <code>\$jsonSchema</code> y TTL	Lecturas con variables opcionales y consultas por cultivo y fecha
Backend	Spring Boot sobre Java 17	Spring Boot 4.1 sobre Java 21 con hilos virtuales	Versiones con soporte vigente
Frontend	React con Vite	PWA en React 19, TypeScript 6 y Tailwind CSS 4	Instalable y con un solo código para teléfono y escritorio



ASPECTO	DISEÑO INICIAL	CONSTRUIDO	POR QUÉ
Infraestructura	Capas gratuitas en la nube	Servidor propio con Docker, Nginx, TLS y despliegue automático desde GHCR	Sin límites de cuota y con control de la seguridad
Inteligencia	<code>evaluarSalud()</code> con reglas fijas; propuesta de LSTM y regresión	Sistema experto, lógica difusa, modelos base, pronóstico Theil-Sen, análisis de flota y aprendizaje continuo con lecturas reales	Explicable desde el primer día y cada vez más preciso con datos reales; una LSTM necesita meses de datos que aún no existen
Notificaciones	Correo y bot en la maceta	PWA y Telegram desde la API, con canales intercambiables, avisos por cultivo, resúmenes y chats compartidos	Un solo lugar con los permisos y datos de cada cuenta
Simulación	Wokwi	Wokwi a mano (cultivo real) y cultivos virtuales siempre encendidos con clima real	Una demo y un QA que no dependen de una pestaña abierta
Seguridad	JWT, AES y límite de peticiones en el acta	Todo lo anterior más control de dueño, cuenta MQTT por cultivo, contenedores endurecidos, CodeQL y SBOM	La revisión encontró rutas sin control de dueño y servicios expuestos

14.1 Organización del código

REPOSITORIO	CONTENIDO
<code>.github</code>	Entornos Docker (demo, desarrollo y producción), Kubernetes, despliegue central, QA, documentación y perfil
SmartPot-API	147
SmartPot-Web	65
SmartPot-AI	84
SmartPot-DataGenerator	34
SmartPot-IoT	15
SmartPot-Broker, -DB, -Cache, -Mail	Imágenes endurecidas de Mosquitto, MongoDB, Redis y Mailpit
SmartPot-Proxy	Archivado: su función la cumple Nginx en el servidor

Convenciones: código e identificadores en inglés; interfaz, mensajes de la API, registros y documentación en español; Python con uv, web con pnpm y Java con Maven Wrapper; Dependabot, CodeQL y CI en todos los repositorios.



14.2 Integración y despliegue continuos

Cada servicio publica su imagen en GHCR (y en Docker Hub como réplica) con SBOM y atestación de procedencia, y pide el despliegue al workflow central, que corre de a uno: si llegan varios pedidos a la vez, solo el más reciente espera. El servidor descarga las imágenes desde GHCR, recrea solo lo que cambió y aplica los esquemas de la base con la migración de SmartPot-DB.

14.3 Inteligencia artificial

La propuesta de investigación planteó pasar de un control reactivo a uno predictivo con modelos entrenados con los datos del propio cultivo. La plataforma lo hace por capas: el sistema experto y la lógica difusa explican cada decisión desde la primera lectura; el pronóstico anticipa tendencias; y el aprendizaje continuo entrena, con las lecturas reales, modelos supervisados que anticipan riego y calor, estiman la humedad del sustrato y reconocen los estados típicos de cada especie.

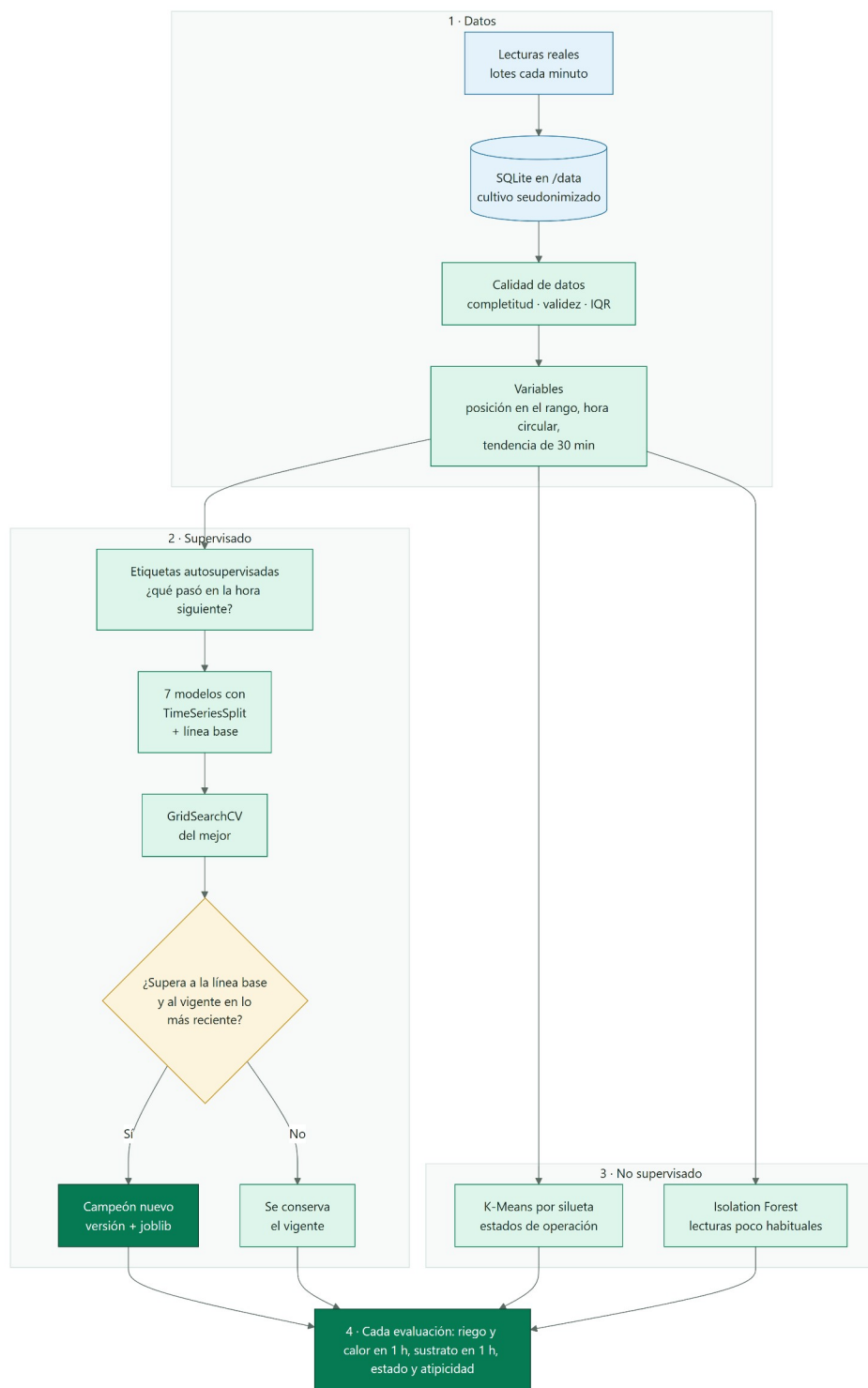


Figura 7 · Aprendizaje continuo



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph prep["1 · Datos"]
        direction LR
        lect["Lecturas reales<br/>lotes cada minuto"] --> alm["SQLite en /data<br/>cultivo
seudonimizado"]
        alm --> cal["Calidad de datos<br/>completitud · validez · IQR"]
        cal --> var["Variables<br/>posición en el rango, hora circular,<br/>tendencia de 30 min"]
    end
    subgraph sup["2 · Supervisado"]
        direction LR
        eti["Etiquetas autosupervisadas<br/>¿qué pasó en la hora siguiente?"] --> cv["7 modelos
con<br/>TimeSeriesSplit<br/>+ línea base"]
        cv --> gs["GridSearchCV<br/>del mejor"]
        gs --> cc["¿Supera a la línea base<br/>y al vigente en lo<br/>más reciente?"]
        cc -->|"Sí"| nuevo["Campeón nuevo<br/>versión + joblib"]
        cc -->|"No"| vig["Se conserva<br/>el vigente"]
    end
    subgraph nosup["3 · No supervisado"]
        direction LR
        km["K-Means por silueta<br/>estados de operación"]
        iso["Isolation Forest<br/>lecturas poco habituales"]
    end
    var --> eti
    var --> km
    var --> iso
    inf["4 · Cada evaluación: riego y calor en 1 h, sustrato en 1 h, estado y atipicidad"]
    nuevo --> inf
    vig --> inf
    km --> inf
    iso --> inf
    classDef data fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef step fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef ask fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef out fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class lect,alm data
    class cal,var,eti,cv,gs,km,iso,vig step
    class cc ask
    class nuevo,inf out

```

14.4 Telegram, de la maceta a la plataforma

La primera versión de SmartPot enviaba las lecturas a un bot de Telegram desde el ESP32. El diseño propuso reemplazarlo por software propio, y así se hizo. Telegram vuelve ahora como **canal de notificación** centralizado en la API: la maceta no conoce Telegram, cada persona vincula su chat con un código de un solo uso y elige qué avisos recibir, y la arquitectura de canales permite sumar otros servicios sin tocar el resto.

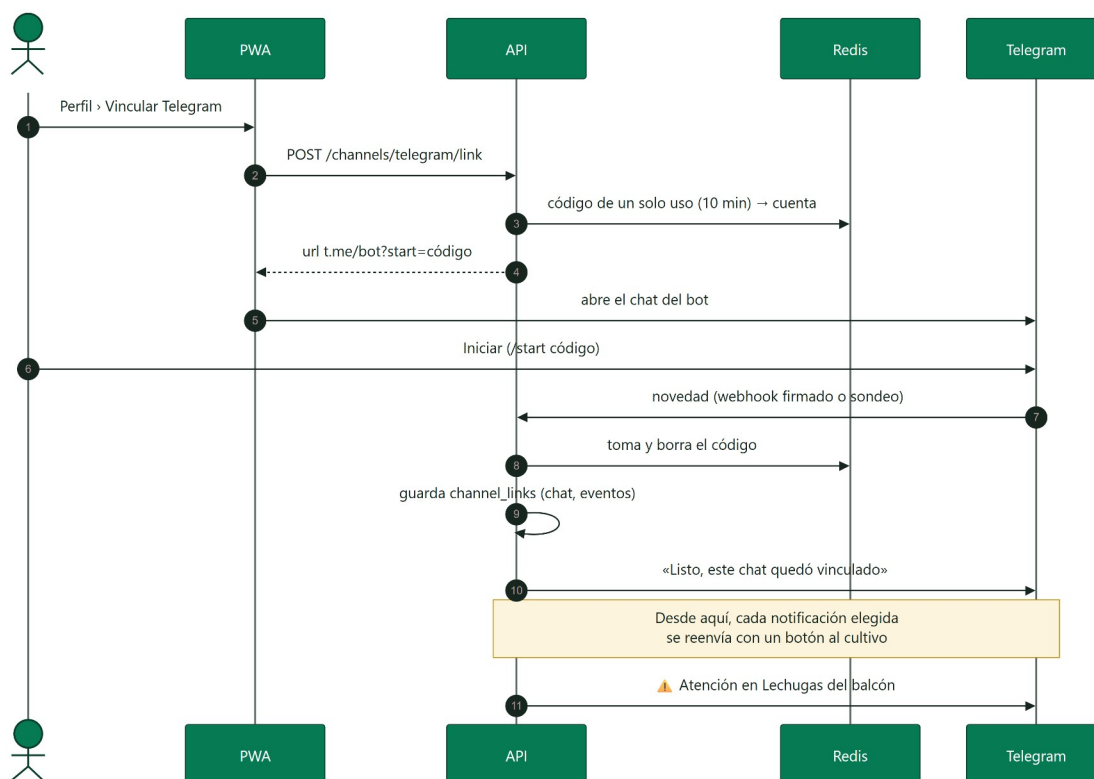


Figura 8 · Vinculación de Telegram

MERMAID



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    actor P as Persona
    participant W as PWA
    participant A as API
    participant C as Redis
    participant T as Telegram
    P->>W: Perfil > Vincular Telegram
    W->>A: POST /channels/telegram/link
    A->>C: código de un solo uso (10 min) → cuenta
    A-->>W: url t.me/bot?start=código
    W->>T: abre el chat del bot
    P->>T: Iniciar (/start código)
    T->>A: novedad (webhook firmado o sondeo)
    A->>C: toma y borra el código
    A->>A: guarda channel_links (chat, eventos)
    A->>T: «Listo, este chat quedó vinculado»
    Note over A,T: Desde aquí, cada notificación elegida<br/>se reenvía con un botón al cultivo
    A->>T: ▲ Atención en Lechugas del balcón

```

14.5 Simulación: Wokwi y cultivos virtuales

Wokwi fue la decisión que permitió enfocarse en el software sin hardware. Sigue siendo la forma de probar el firmware real, a mano y en el navegador. Para una demo y un QA que no dependan de una pestaña abierta, el simulador corre siempre junto a la plataforma: un cultivo virtual sigue medidores manuales, el ciclo del día o el clima real del lugar. La primera versión encendía una «maceta virtual» sobre cualquier cultivo; la revisión separó los dos mundos: el tipo se elige al crear el cultivo y no cambia, Wokwi cuenta como cultivo real, y cada cultivo, real o virtual, se ve en vivo con su forma (maceta, tubos NFT, torre o balsa) y cada actuador.

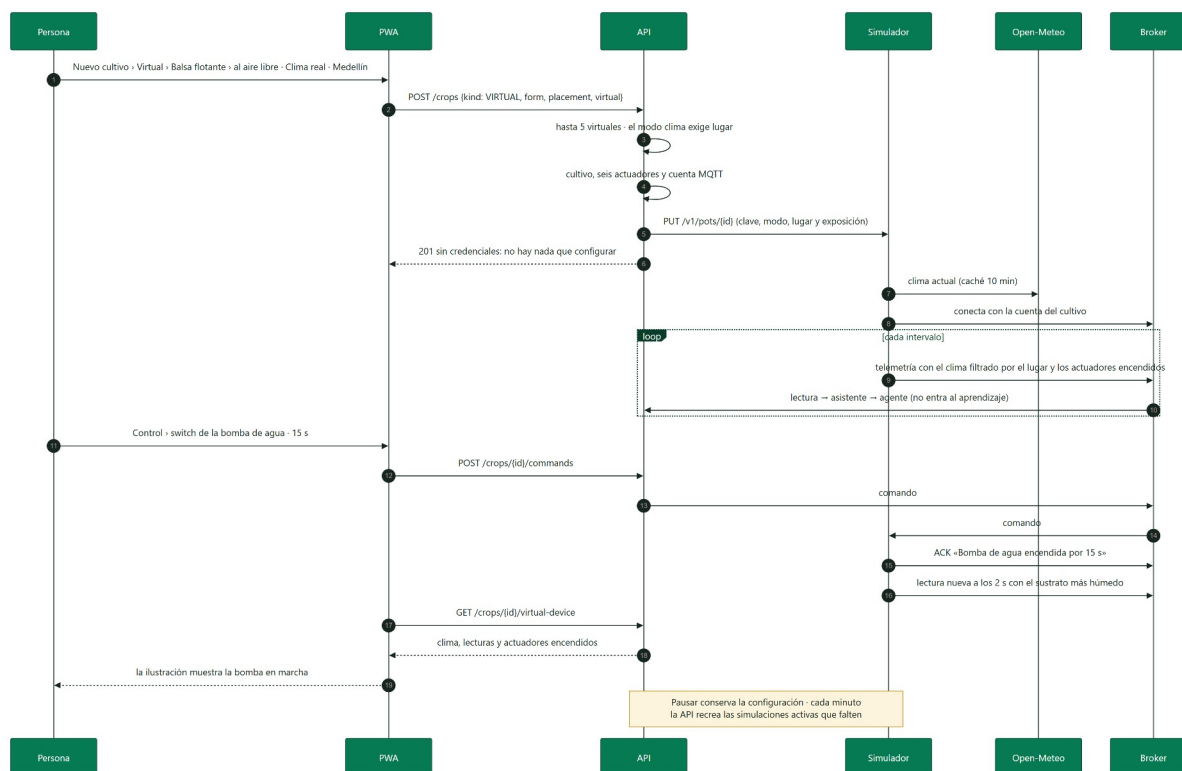


Figura 9 · Cultivo virtual con clima real



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    participant P as Persona
    participant W as PWA
    participant A as API
    participant S as Simulador
    participant O as Open-Meteo
    participant B as Broker
    P->>W: Nuevo cultivo > Virtual > Balsa flotante > al aire libre · Clima real · Medellín
    W->>A: POST /crops {kind: VIRTUAL, form, placement, virtual}
    A->>A: hasta 5 virtuales · el modo clima exige lugar
    A->>A: cultivo, seis actuadores y cuenta MQTT
    A->>S: PUT /v1/pots/{id} (clave, modo, lugar y exposición)
    A-->>W: 201 sin credenciales: no hay nada que configurar
    S->>O: clima actual (caché 10 min)
    S->>B: conecta con la cuenta del cultivo
    loop cada intervalo
        S->>B: telemetría con el clima filtrado por el lugar y los actuadores encendidos
        B->>A: lectura → asistente → agente (no entra al aprendizaje)
    end
    P->>W: Control > switch de la bomba de agua · 15 s
    W->>A: POST /crops/{id}/commands
    A->>B: comando
    B->>S: comando
    S->>B: ACK «Bomba de agua encendida por 15 s»
    S->>B: lectura nueva a los 2 s con el sustrato más húmedo
    W->>A: GET /crops/{id}/virtual-device
    A-->>W: clima, lecturas y actuadores encendidos
    W-->>P: la ilustración muestra la bomba en marcha
    Note over A,S: Pausar conserva la configuración · cada minuto<br/>la API recrea las
    simulaciones activas que falten

```

14.6 El lugar, los switches y los avisos por cultivo

La revisión de la interfaz con cultivos de verdad dejó tres ajustes. El primero es el **lugar**: la persona indica si el cultivo está bajo techo o al aire libre, cuánto sol recibe y en qué ciudad. La ilustración lo dibuja (una ventana con el cielo de afuera, una malla de media sombra o un árbol), el simulador filtra el clima por ese lugar y el asistente compara la luz que pide la especie con la que recibe: deja poner el cultivo donde se quiera, pero recomienda moverlo cuando el lugar ya afecta la salud. El segundo son los **switches**: cada

actuador se maneja con un interruptor que solo acepta órdenes que cambian algo, lo que eliminó los dobles clics y las órdenes repetidas; en los cultivos virtuales cada orden mueve las lecturas en los tres modos y llega una lectura nueva a los dos segundos. El tercero son los **avisos por cultivo**: cada cultivo elige qué avisa por Telegram, al instante o en resúmenes, con un resumen diario y hasta diez chats con los que se comparte; si el servidor no tiene el bot, la aplicación lo muestra en gris con lo que falta.

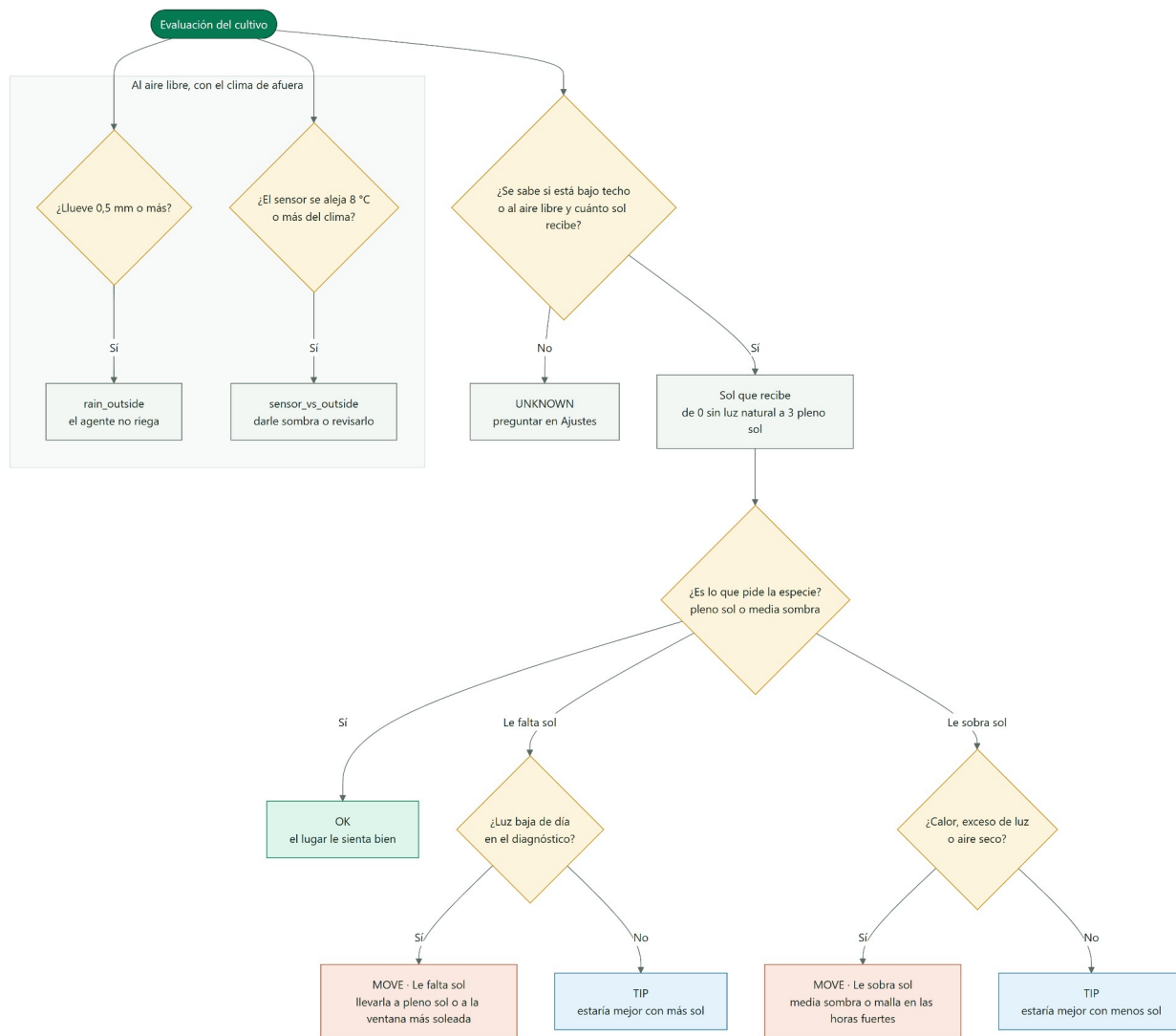


Figura 10 · Consejo de lugar



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    eval(["Evaluación del cultivo"]) --> known{"¿Se sabe si está bajo techo<br/>o al aire libre y cuánto sol recibe?"}
    known -->|"No"| unknown["UNKNOWN<br/>preguntar en Ajustes"]
    known -->|"Sí"| sun["Sol que recibe<br/>de 0 sin luz natural a 3 pleno sol"]
    sun --> fits{"¿Es lo que pide la especie?<br/>pleno sol o media sombra"}
    fits -->|"Sí"| ok["OK<br/>el lugar le sienta bien"]
    fits -->|"Le falta sol"| dark{"¿Luz baja de día<br/>en el diagnóstico?"}
    dark -->|"Sí"| more["MOVE · Le falta sol<br/>llevarla a pleno sol o a la ventana más soleada"]
    dark -->|"No"| tipMore["TIP<br/>estaría mejor con más sol"]
    fits -->|"Le sobra sol"| hot{"¿Calor, exceso de luz<br/>o aire seco?"}
    hot -->|"Sí"| less["MOVE · Le sobra sol<br/>media sombra o malla en las horas fuertes"]
    hot -->|"No"| tipLess["TIP<br/>estaría mejor con menos sol"]
    subgraph outside["Al aire libre, con el clima de afuera"]
        direction LR
        rain{"¿Llueve 0,5 mm o más?"} -->|"Sí"| norain["rain_outside<br/>el agente no riega"]
        gap{"¿El sensor se aleja 8 °C<br/>o más del clima?"} -->|"Sí"|
    end
    check["sensor_vs_outside<br/>darle sombra o revisarlo"]
    end
    eval --> rain
    eval --> gap
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class eval core
    class known,fits,dark,hot,rain,gap sun
    class ok leaf
    class tipMore,tipLess water
    class more,less clay
    class unknown,sun,norain,check muted

```



PARTE V

Pruebas

SECCIÓN 15

Estrategia de pruebas

NIVEL	QUÉ SE PRUEBA	HERRAMIENTAS
Unidad	Reglas de negocio, cifrado, modelos, física del simulador, formato	JUnit y Mockito, pytest, Vitest
Controladores	Cadena de seguridad real, control de dueño, errores en español, webhook firmado	MockMvc con Spring Security
Componentes	Pantallas y componentes de la PWA con su accesibilidad	Testing Library
Contrato	Tópicos y cargas MQTT, esquemas de la IA y del simulador	pytest y pruebas de la API
Imágenes	Autenticación del broker, ACL por cultivo, TLS, validadores de MongoDB, Redis y SMTP	Pruebas de humo en CI
Extremo a extremo	La plataforma completa armada con las ocho imágenes	<code>scripts/e2e.py</code> sobre la demo
Calidad de modelos	Cada modelo aprendido debe superar a una línea base en las lecturas más recientes	Validación cruzada temporal

SECCIÓN 16

Resultados

REPOSITORIO	PRUEBAS
SmartPot-API	106
SmartPot-AI	73
SmartPot-Web	45
SmartPot-DataGenerator	23
SmartPot-IoT	13
SmartPot-Broker	10 comprobaciones
SmartPot-DB, -Cache, -Mail	Pruebas de humo

REPOSITORIO	PRUEBAS
End-to-End	46 comprobaciones

El workflow de QA corre en cada cambio de la organización, cada lunes y a mano; compila las ocho imágenes, levanta la demo y recorre el camino completo de una persona nueva.

16.1 Aceptación de los casos de uso

CASO DE USO	CÓMO SE COMPRUEBA
CU001	E2E: registro, ingreso y rechazo sin sesión
CU002	E2E: cultivo real con credenciales, telemetría con la clave y rechazo de una clave incorrecta; pruebas de la guía de conexión
CU003	E2E: la lectura llega por MQTT, el panel general resume la cuenta y el lugar recibe su consejo; pruebas de la ilustración y de la conexión
CU004	Pruebas del agente y de la API: acciones con modo automático, enfriamiento y falla de sensor
CU005	Pruebas de notificaciones, del reenvío a canales y de los avisos por cultivo
CU006	E2E: series agregadas; pruebas del historial y la exportación
CU007, CU008	Pruebas de controladores y validaciones; E2E: el tipo no cambia y un cultivo real no se simula
CU009	E2E: comando con ACK, switches que rechazan órdenes cruzadas o sin cambio, orden en bloque y automatización en bloque
CU010	E2E: cultivo virtual manual creado sin credenciales, con seis actuadores, que publica por MQTT, informa su estado y se pausa
CU011	Pruebas del bot: código de un solo uso, enlaces para compartir un cultivo, chats privados, estado y escape de HTML
CU012	E2E: la IA recibe lecturas reales; pruebas de la página y de los modelos

16.2 Calidad de los modelos aprendidos

Las pruebas entrenan con dos cultivos simulados durante tres días (2160 lecturas) y exigen que cada modelo supere a su línea base en el 20 % de lecturas más recientes:

TAREA	MEJOR MODELO EN LA VALIDACIÓN	MÉTRICA	MODELO	LÍNEA BASE
¿Se secará el sustrato en 1 h?	Regresión logística	F1 macro	0.89	0.46
¿Habrá calor en 1 h?	Bosque aleatorio	F1 macro	0.92	0.49
Humedad del sustrato en 1 h	Bosque aleatorio	Error medio	1.2 %	4.5 %

**DATOS REALES**

Estos números provienen de series simuladas. En producción cada especie entrena con sus propias lecturas y la página Aprendizaje muestra los puntajes reales; mientras no haya casos suficientes, la tarea queda pendiente y lo explica.

SECCIÓN 17

Estado final y trabajo futuro

PENDIENTE	ORIGEN
Autenticación en dos pasos	RNF-001
Temas y ajustes de visualización	RNF-006
Umbrales propios por cultivo y calibración remota de sensores	RF-008, RF-002
Reportes descargables en documento	RF-012
Actualización en vivo hacia la PWA	RNF-003
Más canales (WhatsApp, correo) sobre la misma interfaz	Arquitectura de canales
Prototipo físico y validación experimental con un ciclo de cultivo	Propuesta de investigación
Redes recurrentes (LSTM) cuando haya meses de datos reales	Propuesta de investigación
Medición formal de cobertura y de latencia	Requisitos de aprobación del acta

SECCIÓN 18

Equipo y créditos

PERSONA	APORTE
Sebastián López Osorno	Gerencia del proyecto, arquitectura y desarrollo de la plataforma
Valentina Restrepo Arboleda	Formulación del proyecto y estudio de mercado
José Alejandro Calderón Jaramillo	Formulación del proyecto y estudio de mercado
Darwin David Castaño Hernández	Propuesta de investigación y casos de uso
Andrés David Medina Martínez	Propuesta de investigación y diagrama de paquetes
Jaziel García Ramírez	Diagramas de componentes y artefactos

El proyecto nació y creció en el Politécnico Colombiano Jaime Isaza Cadavid, con el acompañamiento de los docentes de cada curso.